

Scale-Invariant Baselines Can Certify a Broken Deep Model: Grouped-Normalisation Collapse in Learned Trajectory Screening

Harsha Vardan M Sakamuri, Rohit Michael, Valarmathi Sudhakar

Abstract—Fitting one feature scaler across several groups of data is an ordinary thing to do. We show it can quietly destroy a sequence model while every check a practitioner would normally run says the pipeline is fine. On 70,000 simulated interplanetary trajectories across seven targets, a Transformer trained under a scaler pooled across a group of targets stops responding to its input on Venus: the standard deviation of its held-out prediction is $3.11\text{e-}05$ and its validation AUC is 0.6037, against 0.9999 for the same architecture under per-timestep normalisation. Every one of the 1,500 held-out missions falls inside a window $3.2\text{e-}04$ wide. A gradient-boosted tree fitted to the very same arrays does not notice. its AUC moves by at most 0.0002 across all three normalisation settings, and it reports the task as almost perfectly separable while the network cannot separate it at all. So the usual instinct (check a simple baseline before blaming the data) fails exactly here, because trees are invariant to the thing that breaks the network. The effect is severe but not universal: 1 of 4 targets collapses outright and the rest lose between 0.0003 and 0.0925 AUC while still working, so we report when it happens and when it does not. We separate the two kinds of pooling that are usually confused (pooling across timesteps is survivable, pooling across groups is not), give a diagnostic based on the spread of the predictions rather than on accuracy, and report a second failure of the same model, which cannot reach a rare class that a tree recovers from the identical input. Across 5 seeds the compression is identical and the collapse recurs on 4 of them, so it is a property of the preprocessing and not of one unlucky initialisation.

Index Terms—feature normalisation, shortcut learning, model diagnostics, simulation screening, trajectory analysis, negative results

I. Introduction

Monte Carlo trajectory campaigns spend most of their compute on runs that were already doomed when the spacecraft left its parking orbit. Flight projects run these campaigns as a matter of course; TESS, for one, used a large dispersion campaign to drive its trajectory design [1]. The cost is real enough that a body of work exists purely to make each propagation cheaper [2], [3]. A different and obvious idea is to skip the runs instead: watch the first part of a trajectory and cancel the ones that are going to fail.

We built that screen. Doing so turned up two failures that we think matter well beyond astrodynamics, and one negative result about the screen itself.

The first failure is what this paper is about. A model trained on grouped data, with a single feature scaler fitted

across the whole group, ended up emitting one constant number per group. What makes it worth writing down is not that it happened, but that it went unnoticed for a long time. Validation AUC stayed high, because a mixed validation set lets a model score well by telling groups apart without telling anything apart inside a group. A gradient-boosted baseline run on the same arrays reported near-perfect performance, because trees split on absolute values and simply do not care about the scaling the network depends on. Both of the checks we would have trusted said the pipeline was healthy.

This is uncomfortable, because running a simple baseline is good advice. Tree ensembles are the standard sanity check on tabular and physical data, and they are usually the right one [4]–[6]. The problem is that the property that makes them a good check in general (invariance to how the features are scaled) is precisely what makes them blind to this particular defect. The check cannot fail the model, so it certifies it.

We make three contributions. First, a controlled decomposition of the collapse that separates pooling across timesteps from pooling across groups, and shows only the second is fatal. Second, a demonstration that a scale-invariant baseline actively signs off on the broken configuration, together with a cheap diagnostic that does catch it. Third, a second failure of the same model. It cannot reach a rare class that is recoverable from its own input, which we measure rather than assume. We also report, because it bounds how much the first two matter in practice, that the application which motivated all of this does not need a sequence model at all.

Every number in this paper is produced by a script in the accompanying repository and written to a machine-readable artifact. The paper, including its tables and figures, is generated from those artifacts.

II. Related Work

A. Normalisation and what it assumes

Normalisation is central to training deep networks, and the standard layers (batch, layer, group and instance normalisation) all address some version of the problem [7]–[11]. That how the inputs are scaled decides whether a network trains at all, rather than merely how fast, has been known since well before any of those layers existed [12]. They also all assume something about the input

they are handed: that the feature statistics are either stationary, or at least comparable across whatever the pooling axis is. Fitting one scaler over several groups whose natural ranges differ by orders of magnitude breaks that assumption before the first layer runs. The audio literature ran into a related issue early, where per-recording and per-corpus statistics behave very differently [13]. What we add is a controlled decomposition showing that pooling across groups, not pooling in general, is what does the damage, plus the observation that the resulting model is a constant and not merely a weak one.

B. Shortcut learning and misleading aggregate metrics

Networks prefer easy signals, and will take a shortcut over the intended structure whenever one is available [14], [15]. Related work on Clever-Hans behaviour shows models scoring well for reasons that have nothing to do with the task [16]. Grouped data with heterogeneous scales offers an unusually clean shortcut: the gap between group centroids. A model that learns only that gap can post a validation AUC above 0.95 while being useless inside every individual group, which is exactly what we observed. Our contribution here is a diagnostic (the spread of the predictions within a group) that separates this case from ordinary underfitting and needs no labels. It is also a pipeline-level problem and not a modelling one, of the kind that accumulates in the shared preprocessing code that no single model owns [17].

C. Trees as baselines, and the limits of that

Tree ensembles remain strong on tabular and physical data, often matching or beating deep models [4]–[6], [18], [19], which is a large part of why they are the default baseline. Their splits are axis-aligned and threshold-based, so any monotone rescaling of a feature leaves them unchanged. That invariance is normally a virtue. We point out that it also makes them unable to detect a preprocessing fault that is fatal to a network, and so a practitioner who uses a tree to confirm that the features carry signal gets a true answer to a question they did not mean to ask.

D. Rare classes and optimisation limits

The usual answers to extreme class imbalance are reweighting, focal loss, or resampling [20]–[22], and the effect of imbalance on what a deep classifier actually learns is surveyed at length [23]. The two model families do not respond to it the same way, which is part of why the comparison below is worth making at all [24]. In Section VI we report a case where none of that is the issue: oversampling a rare failure mode by a factor of 19.2 moves its recall not at all, while a tree on the same window separates it almost perfectly. The evidence in the space domain runs the same way. The ESA anomaly benchmark reports that current sequence models still struggle on rare but operationally important events in

real satellite telemetry [25], and rare-anomaly detection on satellite telemetry is an active enough problem to have its own uncertainty-aware methods [26]. Carefully engineered LSTM pipelines do work on this kind of data when the per-channel handling is right [27]. That contrast is the point: the difference is in the engineering, not the model class.

E. Learned screening in astrodynamics

Machine learning is now well established in spacecraft guidance and trajectory design [28], [29], including networks that approximate optimal control directly [30], predict transfer costs without full propagation [31], [32], and learn robust low-thrust policies that are then certified by Monte Carlo [33]. Closer to the data this paper uses, learned models correct orbit predictions from sparse tracking [34] and stand in for Monte Carlo when propagating orbit uncertainty forward [35], and a substantial line of work applies machine learning to spacecraft telemetry for anomaly and fault detection [36], [37], including with the same attention-based architecture family used here [38]. Screening specifically, using a learned model to decide which of a very large number of candidate cases deserve full analysis, is established in conjunction assessment, both as an ESA competition on real conjunction data [39], [40] and as a benchmark of deep approaches against the classical filters they would replace [41]. That last one is the closest analogue to our setting: a screening task on a deterministic astrodynamics simulator, benchmarked against non-deep baselines. Learned early cancellation of individual Monte Carlo trajectory runs, which is what Section VII evaluates, is something we could not find treated in this literature; the closest work reduces the cost of each propagation rather than deciding which propagations to start [2], [3]. Surrogate and early-exit methods for expensive simulation are the general form of the idea [42]–[44].

III. Setup

A. Task and data

The corpus is 70,000 simulated Earth-departure missions across seven interplanetary targets (Mercury, Venus, Mars, Jupiter, Saturn, Uranus and Neptune), with 10,000 missions each, generated with a deterministic two/three-body propagator in NASA’s General Mission Analysis Tool, which is itself an extensively verified piece of software [45]. Each mission is described by six injection offsets: three components of the trans-orbit-insertion burn (dv_V , dv_N , dv_B) and three parking-orbit angles (RAAN, AOP, INC). Each is labelled success or failure, and failures are further typed by physical mode: surface impact, orbit too high, missed target, and a few others.

Telemetry is sampled at a fixed cadence of 54,000 s (15 hours) for the interplanetary transfers and downsampled to roughly 100 steps per mission, so that targets whose flight times differ by two orders of magnitude still produce sequences of comparable length. Thirteen features

are recorded per step, including synodic-frame position, specific orbital energy, flight-path angle, normalised target distance and eccentricity. An eighth target was generated (the Moon) and is deliberately excluded from the study. It is a six-day Earth-centric transfer sampled at 60 s, and shares neither the cost structure of Section VII nor the dynamical regime of a heliocentric transfer.

The screening task is to observe the first 40% of a mission’s trajectory and decide whether to abort it. Splits are 70/15/15 by mission and deterministic in (n, seed). Every experiment draws its partition from a single definition in the repository, so no model is ever scored on data that was used to select it. This is not a hypothetical concern: two earlier analyses in this project were invalid because two scripts independently permuted the same number of missions with the same seed and so produced overlapping ‘test’ sets.

B. Models

The sequence model is a Pre-LN Transformer encoder with `d_model` 128, 8 attention heads, 4 layers and CLS-token pooling, carrying two heads on a shared trunk: one for the mission outcome and one for the failure mode. It is trained on random-length prefixes, so its predictions are in-distribution at any point in the stream, not only at the 40% mark. The baseline is XGBoost with 300 trees at depth 5, fitted to the same normalised prefix, flattened. Throughout this paper, ‘the same input’ means literally the same array, so the baseline consumes the view the network is given, not a re-derived feature set. That detail carries most of the argument, so we are strict about it.

C. The three normalisation settings

Three conditions differ only in how the feature statistics are pooled. Architecture, seed, split, optimiser and epoch budget are held fixed across all of them. Fig. 1 shows what each one estimates its statistics from.

- per-timestep: each feature is standardised against its distribution at that timestep index, across the missions of one target. This is the configuration the working system uses.
- global: one RobustScaler (median and IQR) per target, pooled across all of that target’s timesteps.
- grouped: one RobustScaler pooled across every timestep of every target in a regime group (inner: Mercury, Venus, Mars; outer: Jupiter, Saturn, Uranus, Neptune). This is the setting that failed in production.

It is worth being explicit about why the third setting is not a straw man. Fitting one RobustScaler on a concatenated training set is what you get by following almost any standard pipeline tutorial. Nothing warns you, and the result looks like a correctly normalised array.

IV. Grouped-Normalisation Collapse

A. Why it happens

A mission’s features sweep a wide range over its flight, and different targets occupy ranges that differ by orders of

magnitude. A scaler fitted over a pooled set therefore takes its scale from the largest source of variation in the pool. What the screen actually has to discriminate is neither of those things. It is the spread between missions at a given point in the flight, which is small by comparison. Pooling divides that spread by a denominator chosen for something else, squashing the useful signal toward zero while leaving the between-group structure untouched. The network is then asked to amplify a difference that has been pushed to the edge of what gradient descent can pick up, and Pre-LN layer normalisation removes much of what survives.

We measure this as the signal ratio: the mean across-mission standard deviation of the normalised features, computed inside the observed window only. The restriction matters more than it might seem. Averaged over the whole flight the metric is useless, because late timesteps carry enormous across-mission spread, because failing trajectories have physically diverged by then, and that swamps the early signal, making every condition look healthy. The model commits at 40%, so the quantity that matters is the spread inside the prefix it actually sees.

B. The controlled ablation

The decomposition is the first result. Pooling across timesteps within a single target squashes the signal by one to two orders of magnitude, and yet the network keeps working. It survives on all 4 targets, losing at most a few thousandths of AUC. Pooling across a group is a different animal. On Venus it is catastrophic — held-out output standard deviation $3.11\text{e-}05$, which is to say the network emits one number for every mission regardless of what it is shown, with AUC falling from 0.9999 to 0.6037.

Fig. 2 is the reason we use the word ‘collapse’ rather than ‘underfitting’. A low AUC on its own is ambiguous, being what you would see from a model that has learned something weak or wrong. A held-out prediction distribution three ten-thousandths wide is not ambiguous. The network has stopped being a function of its input.

The second result is that the collapse is selective, and we report it alongside the case that breaks. On the other 3 targets the same manipulation costs between 0.0003 and 0.0925 AUC and leaves the output varying normally. Severity does not follow the signal ratio on its own either. Jupiter’s signal is compressed to 0.0247, lower than Mercury’s 0.0484, and yet Jupiter loses 0.0003 AUC where Mercury loses 0.0925, because Jupiter’s task is separable enough that even a badly attenuated signal is sufficient. Compression is necessary for the collapse but not sufficient; how hard the underlying task is modulates it. A rule of the form ‘it collapses when the ratio drops below X’ is therefore not supported by these data, and we do not claim one. Fig. 3 makes the counter-example visible.

One note on scope. This ablation isolates normalisation and nothing else: a separate model is trained per target, and only the statistics are pooled. The production failure that prompted the study also shared a single model across

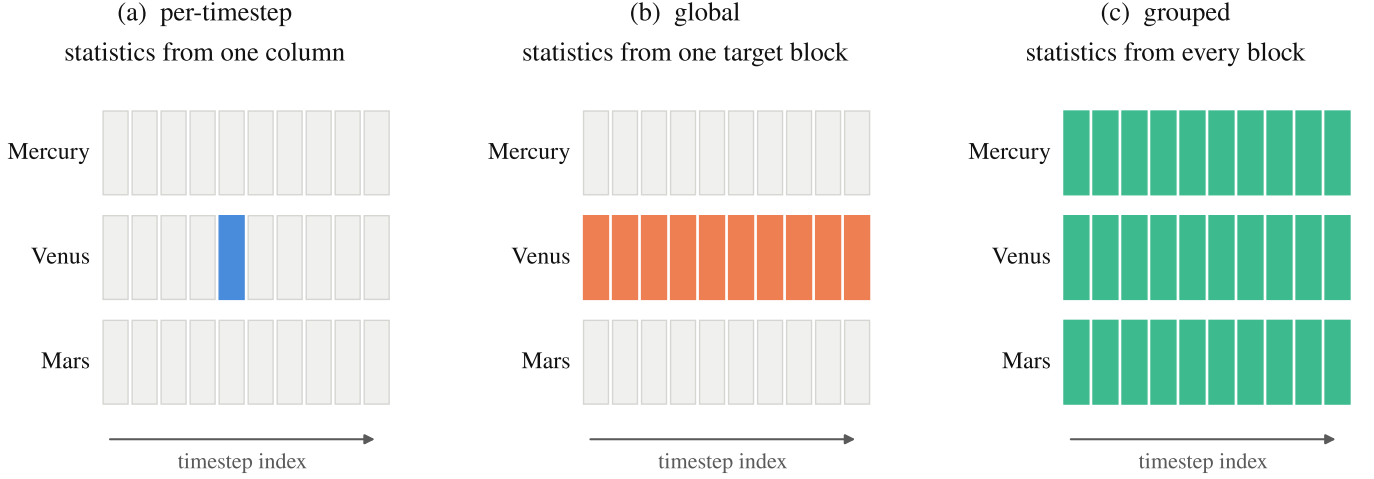


Fig. 1. The three normalisation settings, and the only thing that differs between them: which part of the data the scaler's median and IQR are estimated from. Everything else in the training pipeline is identical.

TABLE I

Normalisation ablation. Identical architecture, seed and split; only the pooling differs. 'sig' is the mean within-timestep standard deviation of the normalised features in the observed window. 'std' is the standard deviation of $P(\text{fail})$ over the held-out split, the quantity that identifies a collapse, as against a model that is merely inaccurate.

Target	per-timestep			global			grouped		
	sig	AUC	std	sig	AUC	std	sig	AUC	std
Venus	0.7681	0.9999	4.72e-01	0.0518	0.9947	4.50e-01	0.0023	0.6037	3.11e-05
Mars	1.0829	0.9994	4.33e-01	0.0472	0.9972	4.00e-01	0.8005	0.9422	4.06e-01
Mercury	0.7686	0.9994	4.27e-01	0.1321	0.9970	4.23e-01	0.0484	0.9069	4.25e-01
Jupiter	0.7708	1.0000	4.78e-01	0.0763	1.0000	4.78e-01	0.0247	0.9997	4.75e-01

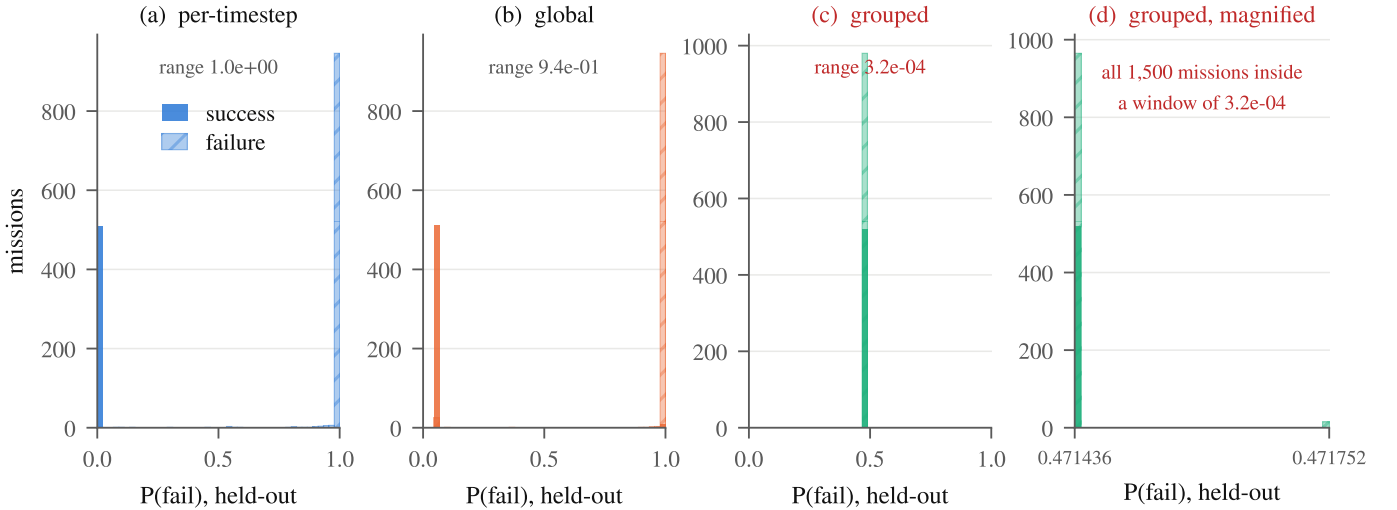


Fig. 2. Held-out $P(\text{fail})$ on Venus under each setting, split by ground truth. Under per-timestep and global normalisation the two populations sit at opposite ends of the axis. Under grouped normalisation every mission receives essentially the same number; panel (d) magnifies that spike and shows the full range of the predictions is about three ten-thousandths wide. This is what distinguishes a collapse from a bad model.

the group, which compounds the problem, since one trunk then has to serve targets whose inputs have all been flattened toward a common constant. The numbers here

are therefore a lower bound on the damage the full configuration does. They establish that pooled normalisation is sufficient on its own to destroy a target, not that this

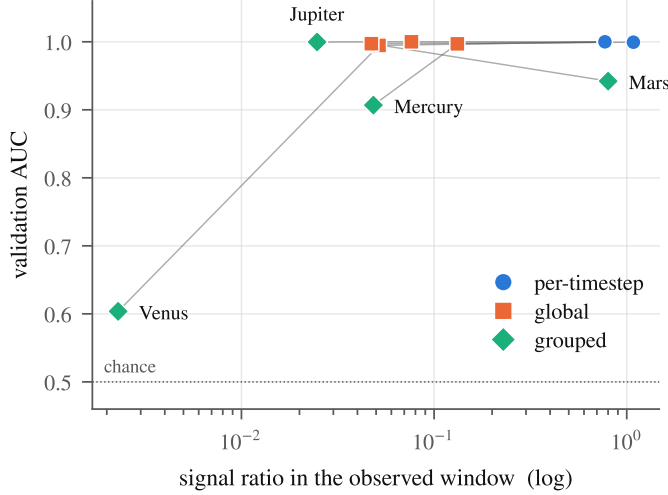


Fig. 3. Signal ratio against validation AUC. If compression alone explained the collapse these points would lie on a curve. Jupiter sits at a lower signal ratio than Mercury and loses almost nothing, so they do not.

TABLE II

XGBoost on the identical normalised window, under each setting. The tree does not react to the preprocessing that destroys the network.

Target	per-timestep	global	grouped	spread
Venus	1.0000	1.0000	1.0000	0.0000
Mars	0.9996	0.9998	0.9998	0.0002
Mercury	0.9997	0.9998	0.9998	0.0001
Jupiter	1.0000	1.0000	1.0000	0.0000

is a complete reconstruction of the original incident.

C. The baseline signs off on the broken configuration

Under the grouped setting the tree reports AUC 1.0000 on Venus, the target where the network has collapsed to a constant at AUC 0.6037. Across all targets the tree’s AUC moves by at most 0.0002 between settings, so for practical purposes it cannot see the choice at all. Someone who runs it to check whether the features carry signal, which is the standard move and the correct instinct, gets an unambiguous yes, and reasonably concludes that the deep model needs tuning rather than that the pipeline is broken.

This is the methodological point, and it is worth stating carefully, because the baseline is not wrong. The information really is present, and a tree really can use it. The baseline is uninformative about the question that was actually being asked, which is whether the network can use it. Invariance to feature scaling, normally an excellent reason to reach for a tree as a diagnostic, is exactly what makes it blind here. The check and the defect are orthogonal by construction, so no amount of care in running the check would have helped.

D. A diagnostic that does catch it

Accuracy on a mixed validation set does not catch the collapse, because a per-group constant ranks the groups



Fig. 4. Both models under grouped normalisation, on the same arrays. The tree is at the ceiling on every target, including the one where the network is barely above chance.

TABLE III

Validation AUC across 5 seeds, mean $\pm$ standard deviation, and the number of seeds on which the grouped run collapsed to a constant output.

Target	per-timestep AUC	grouped AUC	collapsed
Venus	0.9998 ± 0.0001	0.6912 ± 0.1710	4/5
Mars	0.9992 ± 0.0004	0.9457 ± 0.0104	0/5
Mercury	0.9997 ± 0.0002	0.9118 ± 0.0040	0/5
Jupiter	0.9999 ± 0.0002	0.9997 ± 0.0003	0/5

correctly and a pooled metric rewards that. Two checks do catch it:

- Prediction spread within a group. A collapsed model produces almost no variation in its output across inputs. This is a property of the predictions alone, needs no labels, and cleanly separates collapse from ordinary underfitting, which produces predictions that are wrong but still vary.
- Tree against network under identical preprocessing. A large gap in the tree’s favour, with both fed the same array, says the network cannot exploit information that is present, and not that the information is absent.

Both are cheap, and neither requires you to suspect this specific defect in advance, which is the property that makes a diagnostic worth having. We would recommend reporting per-group prediction spread alongside aggregate metrics whenever a model is trained on grouped data with heterogeneous scales. In our case it would have turned a problem that survived several review cycles into a one-line check.

V. Does It Reproduce Across Seeds?

An obvious objection to everything above is that it describes one run. A single collapse at AUC 0.6037 could be an unlucky initialisation. We therefore repeated the full ablation across 5 seeds, each of which redraws the train/validation/test partition as well as the initialisation.

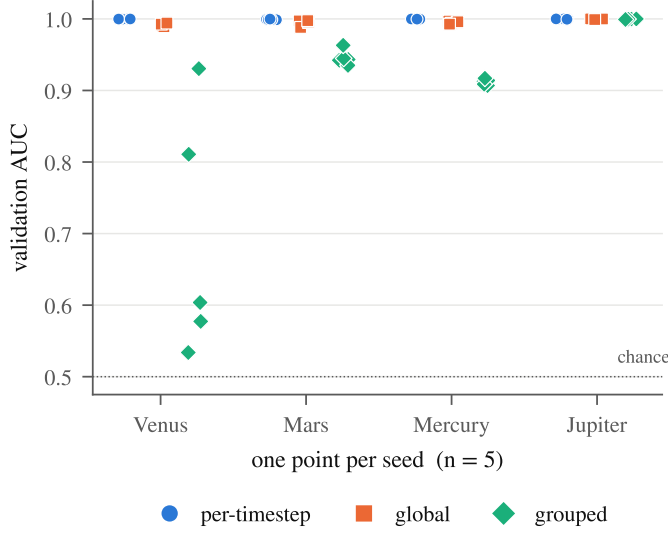


Fig. 5. Every seed plotted individually rather than summarised as an interval. With $n = 5$ the spread of the raw points is the honest summary.

The compression itself does not depend on the seed. Venus’s signal ratio under grouped normalisation lands between 0.00222 and 0.00232 across all 5 runs, a spread of 5% against the 335x compression itself, because it is a property of the preprocessing and the data, not of the optimiser. The healthy condition is stable too. Per-timestep AUC moves by at most 0.0004 between seeds. Mars, Mercury and Jupiter never collapse under any seed.

What does depend on the seed is whether the network escapes. Venus collapses to a constant on 4 of 5 seeds, with held-out prediction spread between $1.57\text{e-}05$ and $4.84\text{e-}05$, four orders of magnitude below the $4.66\text{e-}01$ it reaches under correct normalisation. On the remaining seed it does not fully collapse, but it does not recover either: prediction spread $3.64\text{e-}02$, still an order of magnitude short, at AUC 0.9305 against 0.9999 healthy. So the preprocessing reliably creates the conditions for the collapse, and the collapse is the usual but not the certain outcome.

This is the practical argument for the prediction-spread diagnostic recommended above. Across these runs Venus’s grouped AUC spans 0.5338 to 0.9305, a range that overlaps what a merely-degraded target scores. Mercury sits inside it on every seed while discriminating normally. AUC therefore cannot tell the two states apart. Prediction spread can. The collapsed runs and every healthy run are separated by orders of magnitude with nothing in between. If you are going to watch one number for this failure, watch that one.

The baseline’s blindness replicates more cleanly still. Across 5 seeds, 4 targets and three normalisation settings, 60 fitted trees in total, the tree’s AUC never leaves the range 0.9989 to 1.0000, and within any one seed and target it moves by at most 0.0003 between settings. There is no seed on which the baseline notices. That matters more than the collapse count, because the collapse is one target’s misfortune, but a check that cannot fail is a property of

TABLE IV
Rare-mode oversampling sweep on Uranus. Increasing the sampling weight on the rare mode does not recover it, and does not disturb anything else either.

alpha	resample	test F1	all-mode recall	rare-mode recall
0.00	1.0x	0.9938	0.9877	0.0000
0.50	5.1x	0.9938	0.9877	0.0000
1.00	19.2x	0.9938	0.9877	0.0000

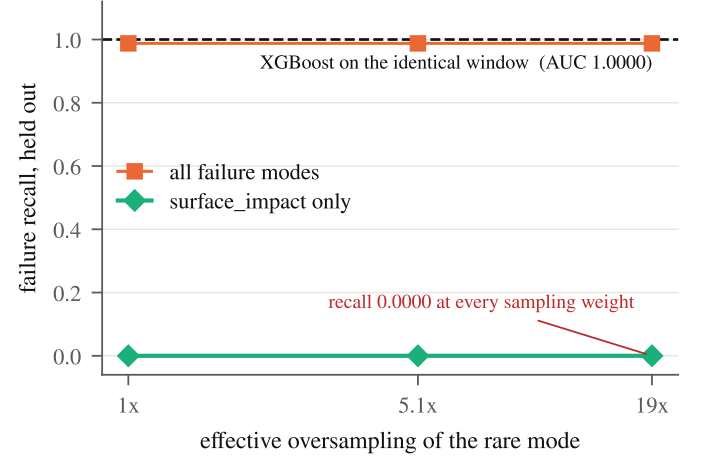


Fig. 6. Recall against oversampling weight. The horizontal line is a tree on the identical window. The rare-mode curve does not move.

the check.

VI. Failing to Reach a Signal That Is Present

A second failure shows up even when the normalisation is correct. On Uranus, the failure mode ‘surface_impact’ accounts for 91 of 4,639 training failures. A tree fitted to the same normalised 40-step window separates that mode from success at AUC 1.0000, with recall 1.0000. The sequence model’s recall on it is 0.0000 at its operating point.

Because both models consume the same array and one of them separates the classes cleanly, the information is present and the sequence model’s failure is an optimisation limit rather than an information limit. Resampling does not close the gap, which rules out the simplest explanation. We are careful not to overclaim the cause: whether it is the loss landscape, the CLS pooling discarding a cue that is localised in time, or the binary head being dominated by the majority mode, we do not know. Probing the trunk representation for linear separability of the rare mode would narrow it down, and that is the next experiment rather than a result we have.

Two denominators need stating, because this number is easy to quote misleadingly. Relative to the majority failure mode the rare mode is oversampled by roughly 45x at the strongest setting. Relative to uniform sampling over the training set, which is what the sampler actually applies, the factor is 19.23x. We report the latter, because it describes what the optimiser saw.

The sweep was repeated across 5 seeds, each of which redraws the split and therefore the rare mode itself. The gap replicates and the extremes do not: sequence recall on the rare mode ranges 0.0000 to 0.1429 across every seed and sampling weight, against a tree on the identical window at recall 0.9333 to 1.0000 and AUC 0.9997 to 1.0000. On every seed the sampling weight makes no difference whatsoever: recall is identical at all three settings within a seed, so the flat line in the figure is not a property of one run.

VII. Where the Screen Actually Belongs

Having gone to the trouble of fixing the sequence model, the honest conclusion is that it should not be used. A six-feature classifier over the injection parameters, evaluated before any propagation happens at all, saves 64.5% of compute against the telemetry model’s 38.3%, at a comparable cost in good missions destroyed. Accuracy is also flat from 10% to 40% observed. The reason is structural rather than a modelling failure: the simulator is deterministic, so the outcome is a fixed function of the injection offsets and the trajectory is their integral. It cannot carry information the parameters do not already have.

This is not an argument that the task is trivial. Logistic regression on the same six features scores between 0.4824 and 0.5410 AUC (chance) on every target, so the map from parameters to outcome is strongly nonlinear and does need a learned model. It just does not need a temporal one. A sequential screen earns its place only where the outcome is not settled at injection: mid-flight stochasticity, unmodelled dynamics, sensor noise, or campaigns where the launch parameters were never recorded. Reinforcement-learning trajectory work that certifies policies through dispersed Monte Carlo campaigns [33] is a natural setting for exactly that.

We include this section because it bounds the practical significance of everything before it. The methodological findings apply to any grouped sequence-modelling pipeline; the application that produced them turns out not to need a sequence model. A reviewer who discovers that on their own is in a worse position than one who is told.

One methodological note on this table, since it cuts against the conclusion we draw. An earlier version was produced under two selection biases: thresholds were fitted on the test labels, and the evaluation set overlapped the sequence model’s validation split. Both flattered T40, the screen this section argues against. Removing them moved the headline by less than 0.1 percentage points, so the negative result survives its own correction, which is the strongest form the claim can take.

VIII. Limitations

- Seeds. Results are replicated across 5 seeds, which is enough to rule out an unlucky initialisation but not enough to support a tight confidence interval. We

report the spread of the individual runs rather than a fitted interval. The collapse itself occurs on 4 of 5; we do not have the sample size to say what governs the exception.

- One domain. The collapse is shown on one corpus. We describe the mechanism in terms of a variance ratio that is not specific to astrodynamics, but we have not reproduced it on a second dataset, and until we do, any predictive threshold on that ratio is a conjecture and not a finding.
- Four targets in the ablation. The collapse is observed on one target out of four. The baseline-blindness result, which is the methodological half, holds on all four.
- Synthetic and deterministic. No execution error, no unmodelled accelerations, no sensor noise, and every mission of a target shares a time base. That shared time base is what makes per-timestep standardisation work at all, and it is a property of the generator and not of real campaigns.
- No real mission data. This is the largest external-validity gap. Public benchmarks of real satellite telemetry now exist [25], [27] and are the obvious next target for the diagnostic.

IX. Conclusion

Sharing a feature scaler across groups whose scales differ can reduce a deep model to a per-group constant, while a tree on the identical arrays reports the task as solved. None of the pieces are exotic. The preprocessing choice is one that a standard pipeline makes by default. The model is a plain Transformer. The baseline is the one most people would reach for. And the standard defence (check a simple baseline) is invariant to the defect by construction, so it cannot fail the broken model.

The fix, per-group normalisation, is not novel and we do not claim it is. The contribution is the detection problem: that a routine check certifies a broken configuration, and that a different check, costing essentially nothing, catches it. Reporting the spread of predictions within a group alongside aggregate metrics would have caught ours immediately.

X. Code and Data Availability

Everything in this paper (the experiment scripts, the JSON artifacts every number is read from, the figure code, and the source of this document) is at <https://github.com/Haise-727/gmat-pred>. A short map of where things are:

- `src/ml/` holds the experiments. `norm_ablation.py` produces Table I, `baseline_invariance.py` Table II, `rare_mode_sweep.py` Table IV, `prune_economics.py` Table V. `splits.py` is the single definition of the train/validation/test partition that all of them share.
- `src/ml/model.py` is the Transformer, and `planet_config.py` the target list, cadences and the recorded reason the Moon is excluded.

TABLE V

Screening economics. Compute is charged in propagation-days, which span a $\sim 100\times$ range across the seven targets. Thresholds are fitted on validation and reported on the untouched test split, so the recall column is measured out of sample rather than achieved by construction.

Screen	Compute saved	Good missions lost	Failure recall
T0: six launch parameters, before propagating	64.5%	0.76%	99.06%
T40: telemetry Transformer at 40%	38.3%	0.21%	98.87%
Cascade: T0 where confident, else T40	65.0%	1.62%	99.94%

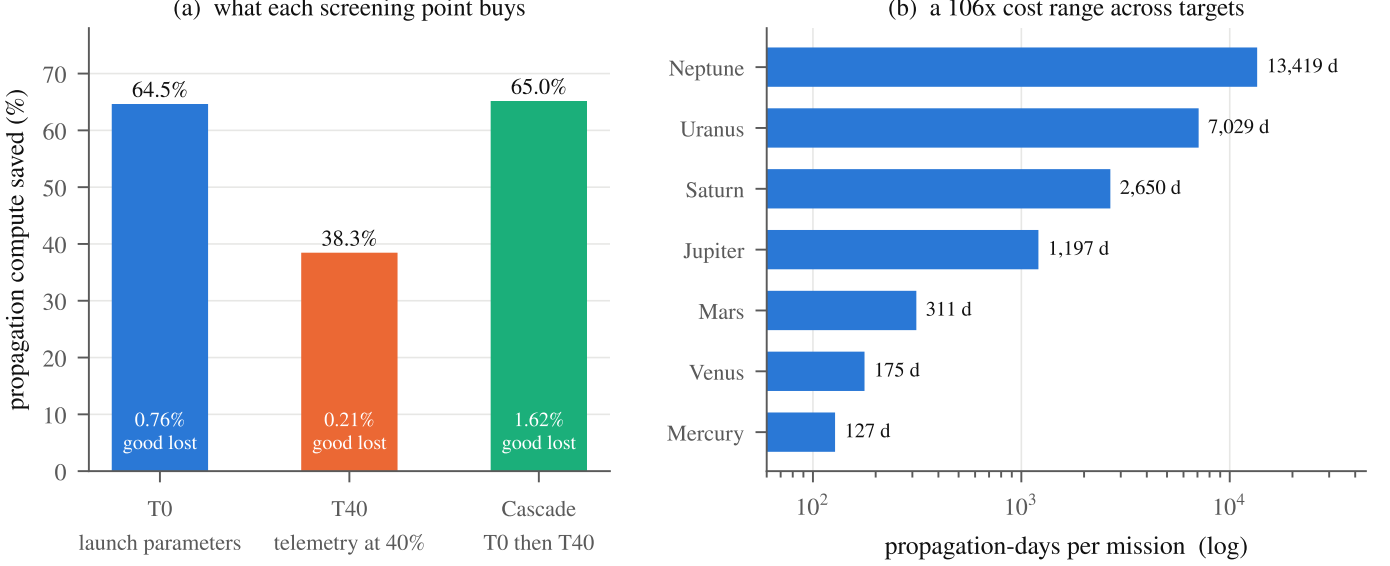


Fig. 7. (a) What each screening point buys and what it costs. (b) Why the weighting is not uniform: per-mission propagation cost spans two orders of magnitude, so the value of a screen is dominated by the outer targets.

- reports/ holds the measured artifacts. Every table and figure here is read from one of these JSON files; multiseed_paper/ holds the per-seed runs behind Table III.
- The manuscript itself is generated from those artifacts rather than written against them: the tables and figures here are produced by script from the JSON above, so no number in this paper is typed in by hand. That build tooling is available from the authors on request.
- docs/RESEARCH_LEDGER.md is the full decision history, including two analyses that were invalidated by a positional join and the corrections that replaced them. docs/LIMITATIONS.md states what the results do not support.

The per-target .npz extracts the experiments read (roughly 70 MB each) are derived from a 71 GB mission table that is too large to host in the repository. The scripts below regenerate the artifacts from those extracts; ORBITGUARD_DATA points at the full table for the economics analysis, which is the only experiment that needs it.

```
export ORBITGUARD_DATA=/path/to/dataset

# Primary results, seed 42
python -m src.ml.norm_ablation          # Table I
python -m src.ml.baseline_invariance     # Table II
python -m src.ml.rare_mode_sweep        # Table IV
python -m src.ml.prune_economics         # Table V
```

```
# Replication, one pass per seed (Table III, Section V)
for S in 0 1 2 3; do
  python -m src.ml.norm_ablation --seed $S \
    --out reports/multiseed_paper/normalisation_ablation_seed$S.
    json
  python -m src.ml.baseline_invariance --seed $S \
    --out reports/multiseed_paper/baseline_invariance_seed$S.json
  python -m src.ml.rare_mode_sweep --seed $S \
    --out reports/multiseed_paper/rare_mode_sweep_seed$S.json
done

# The document
python docs/paper/collect_traces.py      # Fig. 2 data
python docs/paper/figures.py             # all figures
python docs/paper/render_latex.py        # this document
```

References

- [1] C. Nickel, D. J. Dichmann, R. Lebois, S. Lutz, and J. J. K. Parker, “Monte Carlo analysis as a trajectory design driver for the TESS mission,” in AIAA/AAS Astrodynamics Specialist Conference, 2016, AIAA 2016-5665.
- [2] M. Massari, P. Di Lizia, and M. Rasotto, “Nonlinear uncertainty propagation in astrodynamics using differential algebra and graphics processing units,” Journal of Aerospace Information Systems, vol. 14, no. 9, pp. 493–503, 2017.
- [3] B. Jia and M. Xin, “Data-driven surrogate model and admissible-region-based orbital uncertainty propagation,” Journal of Aerospace Information Systems, vol. 19, no. 12, 2022.
- [4] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794.

- [5] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on tabular data?" in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 507–520.
- [6] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Information Fusion*, vol. 81, pp. 84–90, 2022.
- [7] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *International Conference on Machine Learning (ICML)*, 2015, pp. 448–456.
- [8] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.
- [9] Y. Wu and K. He, "Group normalization," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 3–19.
- [10] D. Ulyanov, A. Vedaldi, and V. Lempitsky, "Instance normalization: The missing ingredient for fast stylization," *arXiv preprint arXiv:1607.08022*, 2016.
- [11] L. Huang, J. Qin, Y. Zhou, F. Zhu, L. Liu, and L. Shao, "Normalization techniques in training DNNs: Methodology, analysis and application," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 8, pp. 10 173–10 196, 2023.
- [12] J. Sola and J. Sevilla, "Importance of input data normalization for the application of neural networks to complex industrial problems," *IEEE Transactions on Nuclear Science*, vol. 44, no. 3, pp. 1464–1468, 1997.
- [13] J. Salamon and J. P. Bello, "Deep convolutional neural networks and data augmentation for environmental sound classification," *IEEE Signal Processing Letters*, vol. 24, no. 3, pp. 279–283, 2017.
- [14] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, 2020.
- [15] H. Shah, K. Tamuly, A. Raghunathan, P. Jain, and P. Netrapalli, "The pitfalls of simplicity bias in neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9573–9585.
- [16] S. Lapuschkin, S. Wäldchen, A. Binder, G. Montavon, W. Samek, and K.-R. Müller, "Unmasking Clever Hans predictors and reasoning failures in neural networks via LRP," *Nature Communications*, vol. 10, no. 1, p. 1096, 2019.
- [17] D. K. Chaudhary, S. Srivastava, and V. Kumar, "A review on hidden debts in machine learning systems," in *Second International Conference on Green Computing and Internet of Things (ICGCIoT)*, 2018.
- [18] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 3146–3154.
- [19] J. Pasaribu, N. Yudistira, and W. F. Mahmudy, "Tabular data classification and regression: XGBoost or deep learning with retrieval-augmented generation," *IEEE Access*, vol. 12, pp. 191 719–191 732, 2024.
- [20] M. Buda, A. Maki, and M. A. Mazurowski, "A systematic study of the class imbalance problem in convolutional neural networks," *Neural Networks*, vol. 106, pp. 249–259, 2018.
- [21] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.
- [22] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Class-balanced loss based on effective number of samples," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9268–9277.
- [23] J. Dong, Z. Jiang, D. Pan, Z. Chen, Q. Guan, H. Zhang, G. Gui, and W. Gui, "A survey on confidence calibration of deep learning-based classification models under class imbalance data," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 9, pp. 15 664–15 684, 2025.
- [24] J. Wu, Y. Li, and Y. Ma, "Comparison of XGBoost and the neural network model on the class-balanced datasets," in *IEEE 3rd International Conference on Frontiers Technology of Information and Computer (ICFTIC)*, 2021.
- [25] K. Kotowski, C. Haskamp, J. Andrzejewski, B. Ruszczak, J. Nalepa, D. Lakey, P. Collins, A. Kolmas, M. Bartesaghi, J. Martinez-Heras, and G. De Canio, "European Space Agency benchmark for anomaly detection in satellite telemetry," *arXiv preprint arXiv:2406.17826*, 2024.
- [26] M. A. M. Sadr, Y. Zhu, and P. Hu, "An anomaly detection method for satellites using Monte Carlo dropout," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 59, pp. 2044–2052, 2022.
- [27] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Söderström, "Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2018, pp. 387–395.
- [28] D. Izzo, M. Märten, and B. Pan, "A survey on artificial intelligence trends in spacecraft guidance dynamics and control," *Astrodynamics*, vol. 3, no. 4, pp. 287–299, 2019.
- [29] D. Izzo, E. Blazquez, R. Ferde, S. Origer, C. De Wagter, and G. C. H. E. de Croon, "Optimality principles in spacecraft neural guidance and control," *Science Robotics*, vol. 9, no. 91, p. ead6421, 2024.
- [30] C. Sánchez-Sánchez and D. Izzo, "Real-time optimal control via deep neural networks: Study on landing problems," *Journal of Guidance, Control, and Dynamics*, vol. 41, no. 5, pp. 1122–1135, 2018.
- [31] H. Li, S. Chen, D. Izzo, and H. Baoyin, "Deep networks as approximators of optimal transfers solutions in multitarget missions," *Acta Astronautica*, 2019, arXiv:1902.00250.
- [32] D. Izzo, C. Sprague, and D. Tabor, "Machine learning and evolutionary techniques in interplanetary trajectory design," in *Optimization in Space Engineering*. Springer, 2019, arXiv:1802.00180.
- [33] A. Zavoli and L. Federici, "Reinforcement learning for robust trajectory design of interplanetary missions," *Journal of Guidance, Control, and Dynamics*, vol. 44, no. 8, pp. 1440–1453, 2021.
- [34] B. Li, J. Huang, Y. Feng, F. Wang, and J. Sang, "A machine learning-based approach for improved orbit predictions of LEO space debris with sparse tracking data from a single station," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 56, no. 6, pp. 4253–4268, 2020.
- [35] X. Zhou, D. Qiao, and X. Li, "Neural network-based method for orbit uncertainty propagation and estimation," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 60, no. 1, pp. 1176–1193, 2024.
- [36] S. K. Ibrahim, A. Ahmed, M. A. E. Zeidan, and I. E. Ziedan, "Machine learning methods for spacecraft telemetry mining," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 55, no. 4, pp. 1816–1827, 2019.
- [37] K. R. Naik, A. Holmgren, and J. Kenworthy, "Using machine learning to automatically detect anomalous spacecraft behavior from telemetry data," in *IEEE Aerospace Conference*, 2020.
- [38] S. Jiang, Y. Jiang, Y. Wang, X. Zhang, and Z. Zhang, "Anomaly detection in spacecraft telemetry data based on Transformer-LSTM," in *International Conference on Intelligent Communication and Networking (ICIN)*, 2023.
- [39] T. Uriot, D. Izzo, L. F. Simões, R. Abay, N. Einecke, S. Rebhan, J. Martinez-Heras, F. Letizia, J. Siminski, and K. Merz, "Spacecraft Collision Avoidance Challenge: design and results of a machine learning competition," *arXiv preprint arXiv:2008.03069*, 2020.
- [40] G. Acciarini, F. Pinto, F. Letizia, J. A. Martinez-Heras, K. Merz, C. Bridges, and A. G. Baydin, "Kessler: A machine learning library for spacecraft collision avoidance," in *8th European Conference on Space Debris*, Darmstadt, Germany, 2021.
- [41] E. Stevenson, V. Rodríguez-Fernández, H. Urrutxua, and D. Camacho, "Benchmarking deep learning approaches for all-vs-all conjunction screening," *Advances in Space Research*, vol. 72, no. 7, pp. 2660–2675, 2023.
- [42] C. Baker, M. Peterson, and L. Schmidt, "Accelerating expensive physics-based simulations via machine learning surrogates," *Journal of Computational Physics*, vol. 392, pp. 210–225, 2019.
- [43] H. Sun, Y. Zhang, and B. Liu, "Early-exit deep neural networks for fast inference in simulation pipelines," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 8, pp. 3410–3421, 2021.
- [44] J. Yin, F. Wang, and M. Shankar, "Strategies for integrating deep learning surrogate models with HPC simulation applications," in *IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2022.

- [45] S. P. Hughes, R. H. Qureshi, D. S. Cooley, and J. J. K. Parker, “Verification and validation of the General Mission Analysis Tool (GMAT),” in AIAA/AAS Astrodynamics Specialist Conference, San Diego, CA, 2014, AIAA 2014-4151.